

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT
On

OPERATING SYSTEMS

Submitted by

NEHA KUMARI SAH (1WA24CS185)

In partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

In

COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “OPERATING SYSTEMS – 23CS4PCOPS” carried out by Neha Kumari sah (1WA24CS185), who is Bonafide student of B. M. S. College of Engineering. It is in partial fulfillment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb2026 to June-2026. The Lab report has been approved as it satisfies the academic requirements in respect of a OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Faculty In charge
Seema Patil
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1.	Write a C program to simulate the following non-pre-emptive CPU scheduling algorithm to find turnaround time and waiting time. →FCFS → SJF (pre-emptive & non-pre-emptive) → Priority (pre-emptive & non-pre-emptive) →Round Robin (Experiment with different quantum sizes for RR algorithm)	5
2.	Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.	14
3.	Write a C program to simulate Real-Time CPU Scheduling algorithms: a. Rate- Monotonic b. Earliest-deadline First c. Proportional scheduling	19
4.	Write a C program to simulate producer-consumer problem using semaphores Write a C program to simulate the concept of Dining Philosophers problem.	28
5.	Write a C program to simulate Banker's algorithm for the purpose of deadlock avoidance.	36
6.	Write a C program to simulate the following contiguous memory allocation techniques. a) Worst-fit b) Best-fit c) First-fit	46
7.	Write a C program to simulate page replacement algorithms. a) FIFO b) LRU c) Optimal	53

Course Outcomes

C01	Apply the different concepts and functionalities of Operating System
C02	Analyse various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating System.
C04	Conduct practical experiments to implement the functionalities of Operating system.

Program 1

Write a C program to simulate the following non-pre-emptive CPU scheduling algorithm to find turnaround time and waiting time.

FCFS

Code:

```
#include <stdio.h>

int main() {
    int n,i,j;

    printf("Enter total number of processes: ");
    scanf("%d",&n);

    int pid[n], at[n], bt[n], ct[n], tat[n], wt[n];

    for(i=0;i<n;i++)
    {
        pid[i]=i+1;
        printf("Process %d\n",pid[i]);
        printf("Arrival time: ");
        scanf("%d",&at[i]);    printf("Burst
time: ");    scanf("%d",&bt[i]);
    }

    int temp;

    for(i=0;i<n-1;i++)
    {
        for(j=i+1;j<n;j++)
        {
            if(at[i]>at[j])
            {
                temp=at[i]; at[i]=at[j]; at[j]=temp;
                temp=bt[i]; bt[i]=bt[j]; bt[j]=temp;
                temp=pid[i]; pid[i]=pid[j]; pid[j]=temp;
            }
        }
    }
}
```

```

    int current_time=0;

    for(i=0;i<n;i++)    {
    if(current_time < at[i])
    current_time
    = at[i];

        ct[i] = current_time +
    bt[i];    tat[i] = ct[i] - at[i];    wt[i]
    = tat[i] - bt[i];

        current_time = ct[i];
    }

    float avg_tat=0, avg_wt=0;

    printf("\nProcess ArrivalTime BurstTime CompletionTime TurnaroundTime
    WaitingTime\n");

    for(i=0;i<n;i++)
    {
    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",pid[i],at[i],bt[i],ct[i],tat[i],wt[i]);
    avg_tat += tat[i];    avg_wt += wt[i];
    }

    printf("\nAverage Turnaround Time: %.2f",avg_tat/n);
    printf("\nAverage Waiting Time: %.2f",avg_wt/n);
    return 0;
}

```

Output:

```
C:\Users\Admin\Desktop> gcc sjf.c
Enter the number of processes: 4
Enter arrival time and burst time for process 1: 1 2
Enter arrival time and burst time for process 2: 3 4
Enter arrival time and burst time for process 3: 5 6
Enter arrival time and burst time for process 4: 7 8

PID    AT    BT    CT    TAT    WT
1       1     2     3     2     0
2       3     4     7     4     0
3       5     6    13     8     2
4       7     8    21    14     6

Average Waiting Time: 2.00
Average Turnaround Time: 7.00

Process returned 0 (0x0)   execution time : 13.333 s
Press any key to continue.
```

SJF Code:

```
#include <stdio.h>

int main() {
    int n,i,j;

    printf("Enter number of processes: ");
    scanf("%d",&n);

    int pid[n], at[n], bt[n], ct[n], tat[n], wt[n], finished[n];

    for(i=0;i<n;i++)
    {
        pid[i]=i+1;
        finished[i]=0;
    }

    printf("Enter arrival times:\n");
    for(i=0;i<n;i++)    scanf("%d",&at[i]);
```

```

    printf("Enter burst times:\n");
    for(i=0;i<n;i++)      scanf("%d",&bt[i]);

    int completed=0,current_time=0,min_index;
    float avg_tat=0,avg_wt=0;

    while(completed<n)
    {
        int
min_bt=9999;
min_index=-1;

        for(i=0;i<n;i++)
        {
            if(at[i]<=current_time && finished[i]==0)
            {
                if(bt[i]<min_bt)
                {
                    min_bt=bt[i];
min_index=i;
                }
            }
        }

        if(min_index==-1)
        {
            current_time++;
        }
        else
        {
            ct[min_index]=current_time+bt[min_index];
            tat[min_index]=ct[min_index]-at[min_index];
            wt[min_index]=tat[min_index]-bt[min_index];
            current_time=ct[min_index];
            finished[min_index]=1;      completed++;
        }
    }

    printf("\nSJF scheduling:\n");
    printf("PID\tAT\tBT\tCT\tTAT\tWT\n");

    for(i=0;i<n;i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",pid[i],at[i],bt[i],ct[i],tat[i],wt[i]);
    }

```



```

    avg_tat+=tat[i];    avg_wt+=wt[i];
}

printf("\nAverage turnaround time:%f",avg_tat/n);
printf("\nAverage waiting time:%f",avg_wt/n);

return 0;
}

```

Output:

```

Enter the number of processes: 4
Enter arrival time and burst time for process 1: 1 2
Enter arrival time and burst time for process 2: 3 4
Enter arrival time and burst time for process 3: 5 6
Enter arrival time and burst time for process 4: 7 8

PID    AT    BT    CT    TAT    WT
1       1     2     3     2     0
2       3     4     7     4     0
3       5     6    13     8     2
4       7     8    21    14     6

Average Waiting Time: 2.00
Average Turnaround Time: 7.00

Process returned 0 (0x0)   execution time : 13.333 s
Press any key to continue.

```

Priority (pre-emptive)

Code: #include<stdio.h> int main(){ int n;

```

printf("Enter number of processes: "); scanf("%d",&n);
int pid[n],at[n],bt[n],pr[n],ct[n],tat[n],wt[n],completed[n];
for(int i=0;i<n;i++){ pid[i]=i+1; printf("Enter AT BT
Priority for P%d: ",pid[i]);
scanf("%d%d%d",&at[i],&bt[i],&pr[i]); completed[i]=0;

```

```

} int current_time=0,completed_count=0;
while(completed_count<n){ int
highest_priority=9999,selected=-1; for(int
i=0;i<n;i++){ if(at[i]<=current_time &&
completed[i]==0){ if(pr[i]<highest_priority){
highest_priority=pr[i]; selected=i; }}}
if(selected== -1){ current_time++;
}else{
ct[selected]=current_time+bt[selected];
tat[selected]=ct[selected]-at[selected];
wt[selected]=tat[selected]-bt[selected];
current_time=ct[selected];
completed[selected]=1; completed_count++;
}} printf("PID\tAT\tBT\tPR\tCT\tTAT\tWT\n"); for(int i=0;i<n;i++){
printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",pid[i],at[i],bt[i],pr[i],ct[i],tat[i],wt[i]); }

```

Output:

```

C:\Users\Devi\Desktop\Untitled x + -
Enter number of processes: 3
Enter priority type (1: Higher number = Higher priority, 2: Lower number = Higher priority):
1
Enter Process ID: 1
Enter Arrival Time: 2
Enter Burst Time: 3
Enter Priority: 5
Enter Process ID: 0
Enter Arrival Time: 1
Enter Burst Time: 2
Enter Priority: 3
Enter Process ID: 2
Enter Arrival Time: 4
Enter Burst Time: 6
Enter Priority: 9
PID    AT    BT    PR    CT    TAT    WT
P1     2     3     5     11     9     6
P0     1     2     3     12    11     9
P2     4     6     9     10     6     0
Average Turnaround Time = 8.67
Average Waiting Time = 5.00
Process returned 0 (0x0)   execution time : 31.027 s
Press any key to continue.

```

Priority (Non-pre-emptive)

Code:

```
#include<stdio.h> int main(){ int n;

printf("Enter number of processes: ");

scanf("%d",&n); int

pid[n],at[n],bt[n],pr[n],rt[n],ct[n],tat[n],wt[n];

for(int i=0;i<n;i++){ pid[i]=i+1; printf("Enter

AT BT Priority for P%d: ",pid[i]);

scanf("%d%d%d",&at[i],&bt[i],&pr[i]);

rt[i]=bt[i]; } int

completed=0,current_time=0;

while(completed<n){ int

highest_priority=9999,selected=-1;

for(int i=0;i<n;i++){

if(at[i]<=current_time && rt[i]>0){

if(pr[i]<highest_priority){

highest_priority=pr[i]; selected=i; } }

if(selected!=-1){ current_time++;

} else { rt[selected]--; current_time++;

if(rt[selected]==0){

ct[selected]=current_time;

tat[selected]=ct[selected]-at[selected];

wt[selected]=tat[selected]bt[selected];

completed++; } } }

printf("PID\tAT\tBT\tPR\tCT\tTAT\t

WT\n"); for(int i=0;i<n;i++){

printf("P%d\t%d\t%d\t%d\t%d\t%d\t\t
```

```
%d\n",pid[i],at[i],bt[i],pr[i],ct[i],tat[i],
wt[i]); }
```

Output:

```
C:\Users\Def\Desktop>g++ rr.h
Enter number of processes: 4
Enter Arrival Time for P1: 1
Enter Burst Time for P1: 1
Enter Priority for P1: 2
Enter Arrival Time for P2: 2
Enter Burst Time for P2: 3
Enter Priority for P2: 4
Enter Arrival Time for P3: 1
Enter Burst Time for P3: 0
Enter Priority for P3: 7
Enter Arrival Time for P4: 0
Enter Burst Time for P4: 3
Enter Priority for P4: 5
Process AT    BT    PR    WT    TAT
P1      1      1      2      2      3
P2      2      3      4      2      5
P3      1      0      7      6      6
P4      0      3      5      0      3
Average Waiting Time = 2.50
Average Turnaround Time = 4.25
Process returned 0 (0x0)   execution time : 31.684 s
Press any key to continue.
```

Round Robin (Experiment with different quantum sizes for RR algorithm)

Code:

```
#include<stdio.h> int main(){ int n,tq;
printf("Enter number of processes: ");
scanf("%d",&n); printf("Enter Time
Quantum: "); scanf("%d",&tq); int
pid[n],at[n],bt[n],rt[n],ct[n],tat[n],wt[n];
printf("Enter AT and BT\n"); for(int
i=0;i<n;i++){ pid[i]=i+1;
scanf("%d%d",&at[i],&bt[i]);
rt[i]=bt[i]; } int
completed=0,current_time=0;
while(completed<n){ int
```

```

executed=0; for(int i=0;i<n;i++){
if(at[i]<=current_time && rt[i]>0){
executed=1; if(rt[i]>tq){
current_time+=tq; rt[i]-=tq; }else{
current_time+=rt[i];
ct[i]=current_time; tat[i]=ct[i]-at[i];
wt[i]=tat[i]-bt[i]; rt[i]=0;

completed++; } } }
if(executed==0){ current_time++;
} } printf("PID\tAT\tBT\tCT\tTAT\tWT\n"); for(int i=0;i<n;i++){
printf("P%d\t%d\t%d\t%d\t%d\t%d\n",pid[i],at[i],bt[i],ct[i],tat[i],wt[i]); }

```

Output:

The screenshot shows a terminal window with the following output:

```

Enter number of processes: 3
Enter Arrival Time for P1: 0
Enter Burst Time for P1: 2
Enter Arrival Time for P2: 2
Enter Burst Time for P2: 3
Enter Arrival Time for P3: 4
Enter Burst Time for P3: 6
Enter Time Quantum: 2

```

PID	AT	BT	CT	TAT	WT
P1	0	2	2	2	0
P2	2	3	7	5	2
P3	4	6	11	7	1

```

Average Turnaround Time = 4.67
Average Waiting Time = 1.00
Process returned 0 (0x0)   execution time : 12.020 s
Press any key to continue.

```

Program 2

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue

Code:

```
#include<stdio.h>

#define MAX 20

struct Process {    int pid, at, bt, ct,
tat, wt, type;
};

int main() {    struct Process p[MAX],
sq[MAX], uq[MAX];    int n, i, sq_count = 0,
uq_count = 0, time = 0;    printf("Enter
number of processes: ");    scanf("%d", &n);

    for(i = 0; i < n; i++) {

        printf("\nProcess %d\n", i + 1);

        p[i].pid = i + 1;
        printf("Arrival Time: ");
        scanf("%d", &p[i].at);

        printf("Burst Time: ");    scanf("%d",
&p[i].bt);
```

```

        printf("Type (1=System, 2=User): ");    scanf("%d",
&p[i].type);
        if(p[i].type == 1)
sq[sq_count++] = p[i];    else
uq[uq_count++] = p[i];    }

```

```

        for(i = 0; i < sq_count; i++) {
            if(time <
sq[i].at)        time
= sq[i].at;
            time += sq[i].bt;

            sq[i].ct = time;
        }

```

```

        for(i = 0; i < uq_count; i++) {
if(time < uq[i].at)        time =
uq[i].at;

```

```

        time += uq[i].bt;

```

```

        uq[i].ct = time;
    }

```

```

float total_tat = 0, total_wt = 0;

```

```
printf("\n--- System Queue ---\n");  
printf("PID\tAT\tBT\tCT\tTAT\tWT\n");
```

```
for(i = 0; i < sq_count; i++) {  
    sq[i].tat = sq[i].ct -  
sq[i].at;    sq[i].wt = sq[i].tat  
- sq[i].bt;  
    total_tat +=  
sq[i].tat;    total_wt +=  
sq[i].wt;  
printf("%d\t%d\t%d\t%d\t  
t%d\t%d\n",  
    sq[i].pid,  
sq[i].at,    sq[i].bt,  
sq[i].ct,    sq[i].tat,  
sq[i].wt);  
}
```

```
printf("\n--- User Queue ---\n");  
printf("PID\tAT\tBT\tCT\tTAT\tWT\n");
```

```
for(i = 0; i < uq_count; i++) {  
    uq[i].tat = uq[i].ct -  
uq[i].at;    uq[i].wt = uq[i].tat  
- uq[i].bt;
```



```

        total_tat += uq[i].tat;    total_wt
+= uq[i].wt;

        printf("%d\t%d\t%d\t%d\t%d\t%d\n",
            uq[i].pid,
            uq[i].at,    uq[i].bt,
            uq[i].ct,    uq[i].tat,
            uq[i].wt);
    }
    printf("\nAverage Turnaround Time = %.2f", total_tat / n);
    printf("\nAverage Waiting Time = %.2f\n", total_wt / n);
    printf("\nNeha kumari sah, 1WA24CS185\n");

    return 0;
}

```

Output:

```
"C:\Users\sahne\OneDrive\De  X + v
Neha Kumari Sah (1WA24CS185)
Enter number of processes: 4

Process 1
Arrival Time: 0
Burst Time: 4
Type (1=System,2=User): 1

Process 2
Arrival Time: 0
Burst Time: 3
Type (1=System,2=User): 1

Process 3
Arrival Time: 0
Burst Time: 8
Type (1=System,2=User): 2

Process 4
Arrival Time: 10
Burst Time: 5
Type (1=System,2=User): 1

---System Queue---
PID    AT    BT    CT    TAT    WT
1       0     4     4     4     0
2       0     3     7     7     4
4      10     5    15     5     0

---User Queue---
PID    AT    BT    CT    TAT    WT
3       0     8    23    23    15

Average Turnaround Time=9.75
Average Waiting Time=4.75

Process returned 0 (0x0)  execution time : 31.266 s
Press any key to continue.
```

Program 3

Write a C program to simulate Real-Time CPU Scheduling algorithms:

1) Rate- Monotonic Code:

```
#include<stdio.h>

int gcd(int a,int b){    return
b==0?a:gcd(b,a%b);
}

int lcm(int a,int b){
return (a*b)/gcd(a,b);
}

int main(){

    int n,i,j,time=0;

    printf("Enter the number of processes:");
    scanf("%d",&n);

    int burst[n],period[n],rem[n];

    printf("Enter the CPU burst times:\n");
    for(i=0;i<n;i++)
    scanf("%d",&burst[i]);

    printf("Enter the time periods:\n");
    for(i=0;i<n;i++)
    scanf("%d",&period[i]);

    for(i=0;i<n;i++)
    rem[i]=0;

    int hyper=period[0];

    for(i=1;i<n;i++)
    hyper=lcm(hyper,period[i]);

    printf("LCM=%d\n\n",hyper);

    printf("Rate Monotone Scheduling:\n");
```

```

printf("PID\tBurst\tPeriod\n");

for(i=0;i<n;i++)
printf("%d\t%d\t%d\n",i+1,burst[i],period[i]);

printf("\nScheduling occurs for %d ms\n\n",hyper);

while(time<hyper){

    for(i=0;i<n;i++){
if(time%period[i]==0)
rem[i]=burst[i];
    }

    int idx=-1,minp=9999;

    for(i=0;i<n;i++){
if(rem[i]>0 && period[i]<minp){
minp=period[i];      idx=i;
    }
    }

    if(idx==-1){      printf("%dms onwards: CPU
is idle\n",time);
    }
    else{
        printf("%dms onwards: Process %d running\n",time,idx+1);
rem[idx]--;
    }

    time++;
}

return 0;
}

```

Output:

```
Enter the number of processes:2
Enter the CPU burst times:
20 35
Enter the time periods:
50 100
LCM=100

Rate Monotone Scheduling:
PID      Burst   Period
1         20      50
2         35      100

0.750000 <= 0.828427 => true
Scheduling occurs for 100 ms

0ms onwards: Process 1 running
20ms onwards: Process 2 running
50ms onwards: Process 1 running
70ms onwards: Process 2 running
75ms onwards: CPU is idle

Process returned 0 (0x0)   execution time : 10.631 s
Press any key to continue.
|
```

2) Earliest-deadline First Code:

```
#include <stdio.h>

#define MAX 10

int main() {

    int
    n;
    int burst[MAX], deadline[MAX],
    period[MAX];   int remaining[MAX];   int time
    = 0;
```

```

    printf("Enter number of tasks: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {

        printf("Task %d (burst deadline period): ", i + 1);
        scanf("%d %d %d", &burst[i], &deadline[i], &period[i]);

        remaining[i] = 0;
    }

    printf("\nPID\tBurst\tDeadline\tPeriod\n");

    for (int i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t\t%d\n",
            i + 1,
            burst[i],
            deadline[i],
            period[i]);
    }

    int limit = 20;

    printf("\nScheduling occurs for %d ms\n\n", limit);

    while (time < limit) {

        for (int i = 0; i < n; i++) {

            if (time % period[i] == 0) {
                remaining[i] = burst[i];
            }
        }

        int min_deadline = 9999;
        int selected = -1;

        for (int i = 0; i < n; i++) {

            if (remaining[i] > 0) {

```

```

        if (deadline[i] < min_deadline) {
min_deadline = deadline[i];          selected
= i;
        }
    }
}

    if (selected == -1) {
printf("%dms : CPU is idle.\n", time);
    } else {          printf("%dms : Task
%d is running.\n",
        time,
selected + 1);

        remaining[selected]--;
    }

    time++;
}

return 0;
}

```

Output:

PID	Burst	Deadline	Period
2	2	4	5
1	3	7	20
3	2	8	10

Scheduling occurs for 20 ms

```

0ms : Task 2 is running.
1ms : Task 2 is running.
2ms : Task 1 is running.
3ms : Task 1 is running.
4ms : Task 1 is running.
5ms : Task 3 is running.
6ms : Task 3 is running.
7ms : Task 2 is running.
8ms : Task 2 is running.
9ms : Task 2 is running.
10ms : Task 2 is running.
11ms : Task 3 is running.
12ms : Task 3 is running.
13ms : Task 2 is running.
14ms : Task 2 is running.
15ms : Task 2 is running.
16ms : Task 2 is running.
17ms : Task 1 is running.
18ms : Task 1 is running.
19ms : Task 1 is running.

```

Process returned 0 (0x0) execution time : 0.007 s
Press any key to continue.

|

3) Proportional scheduling

Code:

```
#include<stdio.h>

int main() {
    int n, i, j, total =
    0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int burst[n], share[n];

    printf("Enter Burst Times:\n");
    for(i = 0; i < n; i++) {        scanf("%d",
    &burst[i]);
    }

    printf("Enter Proportional Shares:\n");
    for(i = 0; i < n; i++) {        scanf("%d",
    &share[i]);        total += share[i];
    }

    printf("\nPID\tBurst Time\tShare\n");

    for(i = 0; i < n; i++) {        printf("%d\t%d\t\t%d\n",
    i + 1,
    burst[i],
    share[i]);
    }

    printf("\nProcess Execution Order:\n");

    int completed = 0;

    while(completed < n) {

        completed = 0;

        for(i = 0; i < n; i++) {
```

```

        if(burst[i] > 0)
        {
            int run =
share[i];

            while(run > 0 && burst[i] > 0) {

                printf("Process %d is running\n", i + 1);
burst[i]-;        run--;
            }
        }
        if(burst[i] == 0)
        completed++;
    }
}

    return 0;
}

```

Output:

```
C:\Users\Admin\Desktop\neh X + v
Enter the number of Processes: 3

Process 1:
Tickets: 10

Process 2:
Tickets: 20

Process 3:
Tickets: 30

--- Proportional Share Scheduling ---
Enter the Time Period for scheduling: 5
Tickets picked: 15, Winner: P2
Tickets picked: 43, Winner: P3
Tickets picked: 57, Winner: P3
Tickets picked: 10, Winner: P1
Tickets picked: 22, Winner: P2

The Process: P1 gets 16.67% of Processor Time.

The Process: P2 gets 33.33% of Processor Time.

The Process: P3 gets 50.00% of Processor Time.

Process returned 0 (0x0)   execution time : 8.543 s
Press any key to continue.
|
```

Program 4

Write a C program to simulate producer-consumer problem using semaphores

Code:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <semaphore.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE]; int
in = 0, out = 0;

sem_t empty; sem_t full;

pthread_mutex_t mutex;

void* producer(void* arg)
{   int item, i =
0;

    while (1)
    {       item
= i++;

        sem_wait(&empty);

        pthread_mutex_lock(&mutex);

        buffer[in] = item;

        printf("Produced: %d at buffer[%d]\n", item, in);

        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);

        sem_post(&full);

        sleep(1);
    }
```

```

}

void* consumer(void* arg)
{   int item;

    while (1)
    {
        sem_wait(&full);

        pthread_mutex_lock(&mutex);

        item = buffer[out];

        printf("Consumed: %d from buffer[%d]\n", item, out);

        out = (out + 1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex);

        sem_post(&empty);

        sleep(2);
    }
}

int main() {   printf("Ojashwi Shrestha,
1WA24CS197\n\n");

    pthread_t prod, cons;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);

    pthread_mutex_init(&mutex, NULL);

    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

```

```
    sem_destroy(&empty);  
sem_destroy(&full);  
pthread_mutex_destroy(&mutex);  
  
    return 0;  
}
```

Output:

```
"C:\Users\sahne\OneDrive\De X + v
Enter your choice: 1
Producer has produced: Item 2

Enter 1.Producer 2.Consumer 3.Exit
Enter your choice: 1
Producer has produced: Item 3

Enter 1.Producer 2.Consumer 3.Exit
Enter your choice: 2
Consumer has consumed: Item 1

Enter 1.Producer 2.Consumer 3.Exit
Enter your choice: 2
Consumer has consumed: Item 2

Enter 1.Producer 2.Consumer 3.Exit
Enter your choice: 2
Consumer has consumed: Item 3

Enter 1.Producer 2.Consumer 3.Exit
Enter your choice: 2
Buffer is empty!

Enter 1.Producer 2.Consumer 3.Exit
Enter your choice: 3
Exiting...

Process returned 0 (0x0)   execution time : 9.594 s
Press any key to continue.
```

Write a C program to simulate the concept of **Dining Philosophers** problem.

Code:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define N 5

pthread_mutex_t forks[N]; pthread_t
philosophers[N];

void* philosopher(void* num)
{
    int id =
    *(int*)num;
    int left =
    id;
    int right = (id + 1) % N;

    while (1)
    {
        printf("Philosopher %d is thinking.\n", id);

        sleep(1);

        if (id % 2 == 0)
        {
            pthread_mutex_lock(&forks[left]);

            printf("Philosopher %d picked left fork %d\n",
id,
            left);

            pthread_mutex_lock(&forks[right]);

            printf("Philosopher %d picked right fork %d\n",
id,
            right);
        }
        else
        {
            pthread_mutex_lock(&forks[right]);

            printf("Philosopher %d picked right fork %d\n",
id,
```



```

        right);

        pthread_mutex_lock(&forks[left]);

        printf("Philosopher %d picked left fork %d\n",
id,          left);
    }

    printf("Philosopher %d is eating.\n", id);

    sleep(2);

    pthread_mutex_unlock(&forks[left]);
pthread_mutex_unlock(&forks[right]);

    printf("Philosopher %d put down forks %d and %d\n",    id,
left,          right);
    }

    return NULL;
}

int main()
    int i;
    int ids[N];

    for (i = 0; i < N; i++)
    {
        pthread_mutex_init(&forks[i], NULL);

        ids[i] = i;
    }

    for (i = 0; i < N; i++)
    {

pthread_create(&philosophers[i],
NULL,          philosopher,
                &ids[i]);
    }

    for (i = 0; i < N; i++)

```

```
    {  
        pthread_join(philosophers[i], NULL);  
    }  
  
    for (i = 0; i < N; i++)  
    {  
        pthread_mutex_destroy(&forks[i]);  
    }  
  
    return 0;  
}
```

Output:

```
"C:\Users\sahne\OneDrive\De X + v
Philosopher 0 is thinking.
Philosopher 2 is thinking.
Philosopher 1 is thinking.
Philosopher 3 is thinking.
Philosopher 4 is thinking.
Philosopher 1 picked up right fork 2.
Philosopher 1 picked up left fork 1.
Philosopher 1 is eating.
Philosopher 3 picked up right fork 4.
Philosopher 3 picked up left fork 3.
Philosopher 3 is eating.
Philosopher 0 picked up left fork 0.
Philosopher 3 put down forks 3 and 4.
Philosopher 0 picked up right fork 1.
Philosopher 0 is eating.
Philosopher 4 picked up left fork 4.
Philosopher 2 picked up left fork 2.
Philosopher 2 picked up right fork 3.
Philosopher 2 is eating.
Philosopher 3 is thinking.
Philosopher 1 put down forks 1 and 2.
Philosopher 1 is thinking.
Philosopher 1 picked up right fork 2.
Philosopher 1 picked up left fork 1.
Philosopher 1 is eating.
Philosopher 4 picked up right fork 0.
Philosopher 4 is eating.
Philosopher 0 put down forks 0 and 1.
Philosopher 0 is thinking.
Philosopher 2 put down forks 2 and 3.
Philosopher 2 is thinking.
Philosopher 3 picked up right fork 4.
Philosopher 3 picked up left fork 3.
Philosopher 3 is eating.
Philosopher 4 put down forks 4 and 0.
Philosopher 4 is thinking.
Philosopher 2 picked up left fork 2.
Philosopher 1 put down forks 1 and 2.
Philosopher 1 is thinking.
Philosopher 0 picked up left fork 0.
Philosopher 0 picked up right fork 1.
Philosopher 0 is eating.
Philosopher 3 put down forks 3 and 4.
```

Program 5

Write a C program to simulate Banker's algorithm for the purpose of deadlock avoidance.

Banker's Algorithm Code:

```
#include <stdio.h>

int main() {
    int n, m, i, j, k;

    printf("Enter number of processes: ");    scanf("%d",
    &n);

    printf("Enter number of resources: ");
    scanf("%d", &m);

    int alloc[n][m], max[n][m], need[n][m];
    int avail[m], finish[n], safeSeq[n];

    printf("Enter Allocation Matrix:\n");

    for(i = 0; i < n; i++)
    {
        for(j = 0; j < m;
        j++)
        {
            scanf("%d", &alloc[i][j]);
        }
    }

    printf("Enter Maximum Demand Matrix:\n");
```

```

    for(i = 0; i < n; i++)
    {
        for(j = 0; j < m;
j++)
        {
            scanf("%d", &max[i][j]);
        }
    }

```

```

printf("Enter Available Resources:\n");

```

```

    for(i = 0; i < m; i++)
    {
        scanf("%d",
&avail[i]);
    }

```

```

    for(i = 0; i < n; i++)
    {
        for(j = 0; j < m;
j++)
        {
            need[i][j] = max[i][j] - alloc[i][j];
        }
    }

```

```

    for(i = 0; i < n; i++)
        finish[i]
= 0;

```

```

    int count = 0;

```

```

while(count < n)
{
    int found
= 0;

    for(i = 0; i < n; i++)
    {
        if(finish[i] ==
0)
        {
            for(j =
0; j < m; j++)
            {
                if(need[i][j]
> avail[j])
                    break;
            }
            if(j == m)
            {
                for(k = 0; k <
m; k++)
                    avail[k] +=
alloc[i][k];

                safeSeq[count++] = i;
                finish[i] =
1;
                found =
1;
            }
        }
    }

    if(found == 0)
    {
        printf("System is not in safe
state\n");
        return 0;
    }
}

```

```

    }

    printf("\nSystem is in a safe state.\n");

    printf("Safe sequence is: ");

    for(i = 0; i < n; i++)
    {
        printf("P%d",
safeSeq[i]);
        if(i != n - 1)
            printf("
-> ");
    }
    return 0; } Output:

```

```
C:\Users\admin\Desktop\safe X + v
Enter number of processes: 5
Enter number of resources: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Maximum Demand Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Resources:
3 3 2

System is in a safe state.
Safe sequence is: P1 -> P3 -> P4 -> P0 -> P2

Process returned 0 (0x0)   execution time : 64.936 s
Press any key to continue.
|
```

Deadlock Detection Code:

```
#include <stdio.h>
```

```
int main()
```

```
    int n, m, i, j, k;
```

```
    printf("Enter the  
    number of
```



```
processes: ");    scanf("%d",  
&n);
```

```
    printf("Enter the number of resources: ");    scanf("%d",  
&m);
```

```
    int alloc[n][m], request[n][m];    int  
    avail[m], finish[n], safeSeq[n];
```

```
    printf("Enter the allocation matrix:\n");
```

```
    for(i = 0; i < n; i++)  
    {        printf("Process %d:  
", i);
```

```
        for(j = 0; j < m; j++)  
        {  
            scanf("%d", &alloc[i][j]);  
        }  
    }
```

```
    printf("Enter the request matrix:\n");
```

```
    for(i = 0; i < n; i++)  
    {        printf("Process %d:  
", i);        for(j = 0; j < m; j++)  
        {
```

```

        scanf("%d", &request[i][j]);
    }
}

printf("Enter the available resources: ");

for(i = 0; i < m; i++)
{
    scanf("%d",
&avail[i]);
}

for(i = 0; i < n; i++)    finish[i]
= 0;

int count = 0;

while(count < n)
{
    int found
= 0;

    for(i = 0; i < n; i++)
    {
        if(finish[i] == 0)
        {
            for(j = 0; j <
m; j++)    {
                if(request[i][j] > avail[j])
                    break;

```

```

    }
    if(j == m)
    {
        for(k = 0; k <
m; k++)
        avail[k] +=
alloc[i][k];

```

```

        safeSeq[count++] = i;
    finish[i] =
    1;
    found =
    1;
    }
    }
    }
    if(found ==
0)
        break;
    }

```

```

    int deadlock = 0;

    for(i = 0; i < n; i++)
    {
        if(finish[i] == 0)
        {
            deadlock = 1;

```

```

        printf("Process P%d is deadlocked\n", i);

```

```

    }
}

if(deadlock == 0) {
printf("\nSystem is in safe
state.\n");

printf("Safe Sequence is: ");

for(i = 0; i < count; i++)
{
printf("P%d ", safeSeq[i]);
} }
return 0;
}

```

Output:

```
C:\Users\admin\Desktop\neh: X + v
Enter the number of processes: 5
Enter the number of resources: 3
Enter the allocation matrix:
Process 0: 0 1 0
Process 1: 2 0 0
Process 2: 3 0 3
Process 3: 2 1 1
Process 4: 0 0 2
Enter the request matrix:
Process 0: 0 0 0
Process 1: 2 0 2
Process 2: 0 0 0
Process 3: 1 0 0
Process 4: 0 0 2
Enter the available resources: 0 0 0

System is in safe state.
Safe Sequence is: P0 P2 P3 P4 P1

Process returned 0 (0x0)   execution time : 70.810 s
Press any key to continue.
```

Program 6

Write a C program to simulate the following contiguous memory allocation techniques. a)

Worst-fit

b) Best-fit

c) First-fit

Code:

```
#include <stdio.h>
```

```
int main() {
```

```
    printf("Ojashwi Shrestha, 1WA24CS197\n\n");
```

```
    int m, n, i, j;
```

```
    printf("Enter number of memory blocks: ");    scanf("%d",
&m);
```

```
    int blockSize[m], tempBlock[m];
```

```
    printf("Enter sizes of %d memory blocks:\n", m);
```

```
    for(i = 0; i < m; i++)    scanf("%d",
&blockSize[i]);
```

```
    printf("Enter number of processes: ");    scanf("%d",
&n);
```

```
    int processSize[n];
```

```

printf("Enter sizes of %d processes:\n", n);

for(i = 0; i < n; i++)
scanf("%d", &processSize[i]);

int allocation[n];

for(i = 0; i < m; i++)    tempBlock[i]
= blockSize[i];

for(i = 0; i < n; i++) {

    allocation[i] = -1;

    for(j = 0; j < m; j++) {

        if(tempBlock[j] >= processSize[i]) {
allocation[i] = j;

            tempBlock[j] -= processSize[i];

break;
        }
    }

    printf("\n--- First Fit ---\n");

```

```

printf("Process No.\tProcess Size\tBlock No.\n");

for(i = 0; i < n; i++) {
    if(allocation[i] != -
1)
printf("%d\t\t%d\t\t%d\n",
        i + 1,        processSize[i],
allocation[i] + 1);    else        printf("%d\t\t%d\t\tNot
Allocated\n",
        i + 1,
processSize[i]);
}

for(i = 0; i < m; i++)    tempBlock[i]
= blockSize[i];

for(i = 0; i < n; i++) {

    int best = -1;

    for(j = 0; j < m; j++) {

        if(tempBlock[j] >= processSize[i]) {

            if(best == -1 || tempBlock[j] < tempBlock[best])
                best = j;

        }
    }
}

```



```

    }

    if(best != -1) {

        allocation[i] = best;

        tempBlock[best] -= processSize[i];
    }
else {

    allocation[i] = -1;
}
}

printf("\n--- Best Fit ---\n");

printf("Process No.\tProcess Size\tBlock No.\n");

for(i = 0; i < n; i++) {

    if(allocation[i] != -1)
printf("%d\t\t%d\t\t%d\n",          i + 1,
processSize[i],
allocation[i] + 1);    else    printf("%d\t\t%d\t\tNot
Allocated\n",
                i + 1,
processSize[i]);
}

```

```

    for(i = 0; i < m; i++)      tempBlock[i]
= blockSize[i];

    for(i = 0; i < n; i++) {

        int worst = -1;

        for(j = 0; j < m; j++) {

            if(tempBlock[j] >= processSize[i]) {

                if(worst == -1 || tempBlock[j] > tempBlock[worst])
                    worst = j;
            }
        }

        if(worst != -1) {

            allocation[i] = worst;

            tempBlock[worst] -= processSize[i];
        }
    else {

        allocation[i] = -1;
    }
}

```

```

printf("\n--- Worst Fit ---\n");

printf("Process No.\tProcess Size\tBlock No.\n");

for(i = 0; i < n; i++) {
    if(allocation[i] != -
1)
printf("%d\t\t%d\t\t%d\n",
        i + 1,        processSize[i],
allocation[i] + 1);    else        printf("%d\t\t%d\t\tNot
Allocated\n",
        i + 1,
processSize[i]);
    }

return 0;
}

```

Output:

```
"C:\Users\sahne\OneDrive\De X + v
Enter sizes of memory blocks:
100 500 200 300 600
Enter number of processes: 4
Enter sizes of processes:
212 417 112 426

FIRST FIT Allocation:
Process No    Process Size    Block No
1             212          2
2             417          5
3             112          2
4             426        Not Allocated

BEST FIT Allocation:
Process No    Process Size    Block No
1             212          4
2             417          2
3             112          3
4             426          5

WORST FIT Allocation:
Process No    Process Size    Block No
1             212          5
2             417          2
3             112          5
4             426        Not Allocated

Process returned 0 (0x0)    execution time : 51.396 s
Press any key to continue.
```

Program 7

Write a C program to simulate page replacement algorithms.

a) FIFO

b) LRU

c) Optimal **Code:** #include <stdio.h> #include <stdlib.h>

```
void fifo(int pages[], int n, int frames)
```

```
{   int frame[frames];   int i, j, k =  
0, page_faults = 0;
```

```
    for(i = 0; i < frames; i++)       frame[i]  
= -1;
```

```
    printf("\n--- FIFO Page Replacement ---\n");
```

```
    for(i = 0; i < n; i++)  
    {       int found  
= 0;
```

```
        for(j = 0; j < frames; j++)  
    {       if(frame[j] == pages[i])  
        {  
found = 1;  
break;  
        }  
    }
```

```

        if(!found)    {
frame[k] = pages[i];

        k = (k + 1) % frames;

        page_faults++;
    }

    printf("Page %d -> [", pages[i]);

    for(j = 0; j < frames; j++)        printf("%d
", frame[j]);

    printf("]\n");
}

printf("Total Page Faults (FIFO): %d\n", page_faults);
}

void lru(int pages[], int n, int frames)
{
    int frame[frames],
time[frames];    int i, j,
page_faults = 0, t = 0;

    for(i = 0; i < frames; i++)
    {
        frame[i]
= -1;        time[i]

```

```

= 0;
    }

    printf("\n--- LRU Page Replacement ---\n");

    for(i = 0; i < n; i++)
    {
        int found
= 0;

        for(j = 0; j < frames; j++)
        {
            if(frame[j] == pages[i])
            {
found = 1;

t++;

                time[j] =
t;
            break;
        }
    }

    if(!found)
    {
        int min_time = time[0], pos = 0;

        for(j = 1; j < frames; j++)
        {
            if(time[j] < min_time)

```

```

        {
min_time = time[j];          pos
= j;
        }
    }

    frame[pos] = pages[i];

t++;

    time[pos] = t;

    page_faults++;
}

printf("Page %d -> [", pages[i]);

    for(j = 0; j < frames; j++)
printf("%d ", frame[j]);

    printf("]\n");
}

printf("Total Page Faults (LRU): %d\n", page_faults);
}

void optimal(int pages[], int n, int frames)

```



```

{   int frame[frames], i, j, page_faults =
0;

    for(i = 0; i < frames; i++)        frame[i]
= -1;

    printf("\n--- Optimal Page Replacement ---\n");

    for(i = 0; i < n; i++)
    {        int found
= 0;

        for(j = 0; j < frames; j++)
    {        if(frame[j] == pages[i])
            {                found
= 1;                break;
            }
        }

        if(!found)
        {                int replace_index = -1,
farthest = i;

            for(j = 0; j < frames; j++)
            {                int
k;

```

```

        for(k = i + 1; k < n; k++)
        {
            if(frame[j]
== pages[k])
            {
                if(k > farthest)
                {
                    farthest = k;
replace_index = j;
                }

break;
            }
        }
        if(k == n)
        {
replace_index = j;
break;
        }
    }

    if(replace_index == -1)        replace_index
= 0;

    frame[replace_index] = pages[i];
page_faults++;
}

printf("Page %d -> [", pages[i]);

```

```

        for(j = 0; j < frames; j++)        printf("%d
", frame[j]);

        printf("]\n");
    }

    printf("Total Page Faults (Optimal): %d\n", page_faults);
}

int main() {        int n, frames;

    printf("Enter number of pages: ");

    scanf("%d", &n);

    int pages[n];

    printf("Enter page reference string: ");

    for(int i = 0; i < n; i++)
        scanf("%d", &pages[i]);

    printf("Enter number of frames: ");
    scanf("%d", &frames);

    fifo(pages, n, frames);

    lru(pages, n, frames);

```

```
    optimal(pages, n, frames);
```

```
    return 0;
```

```
}
```

Output:

```

Enter number of pages: 12
Enter page reference string:
7 0 1 2 0 3 0 4 2 3 0 3
Enter number of frames: 3

--- FIFO Page Replacement ---
Page 7 -> [7 ]
Page 0 -> [7 0 ]
Page 1 -> [7 0 1 ]
Page 2 -> [2 0 1 ]
Page 0 -> [2 0 1 ]
Page 3 -> [2 3 1 ]
Page 0 -> [2 3 0 ]
Page 4 -> [4 3 0 ]
Page 2 -> [4 2 0 ]
Page 3 -> [4 2 3 ]
Page 0 -> [0 2 3 ]
Page 3 -> [0 2 3 ]
Total Page Faults (FIFO): 10

--- LRU Page Replacement ---
Page 7 -> [7 ]
Page 0 -> [7 0 ]
Page 1 -> [7 0 1 ]
Page 2 -> [2 0 1 ]
Page 0 -> [2 0 1 ]
Page 3 -> [2 0 3 ]
Page 0 -> [2 0 3 ]
Page 4 -> [4 0 3 ]
Page 2 -> [4 0 2 ]
Page 3 -> [4 3 2 ]
Page 0 -> [0 3 2 ]
Page 3 -> [0 3 2 ]
Total Page Faults (LRU): 9

--- Optimal Page Replacement ---
Page 7 -> [7 ]
Page 0 -> [7 0 ]
Page 1 -> [7 0 1 ]
Page 2 -> [2 0 1 ]
Page 0 -> [2 0 1 ]
Page 3 -> [2 0 3 ]
Page 0 -> [2 0 3 ]
Page 4 -> [2 4 3 ]
Page 2 -> [2 4 3 ]
Page 3 -> [2 4 3 ]
Page 0 -> [0 4 3 ]
Page 3 -> [0 4 3 ]
Total Page Faults (Optimal): 7

Process returned 0 (0x0)   execution time : 34.195 s

```